UNIVERSITY OF LIMERICK
OLLSCOIL LUIMNIGH
Department of Computer Science and Information Systems

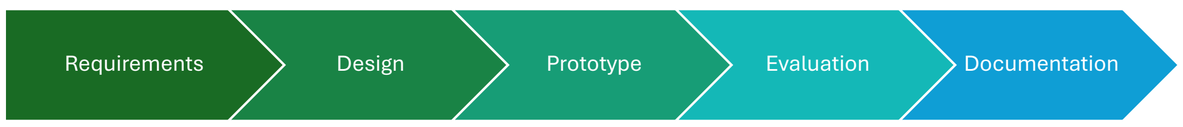
Event-Driven Architecture for a Real-Time Research Management System (RMS)

Rohan Sikder
University of Limerick

Introduction

Design a modular and scalable **Research Management System (RMS)** using **Event-Driven Architecture (EDA)**.
Enable real-time collaboration through asynchronous event communication (threads, mentions, notifications).
Demonstrate frontend-driven architecture without relying on a backend server during the prototype phase.
Evaluate system performance, usability and maintainability using tools like Lighthouse, SonarQube and ESLint.
Justify architectural decisions based on quality attributes, user needs and scalability requirements.

Method



Requirements

Enable real-time collaboration with thread creation, replies, mentions and notifications.

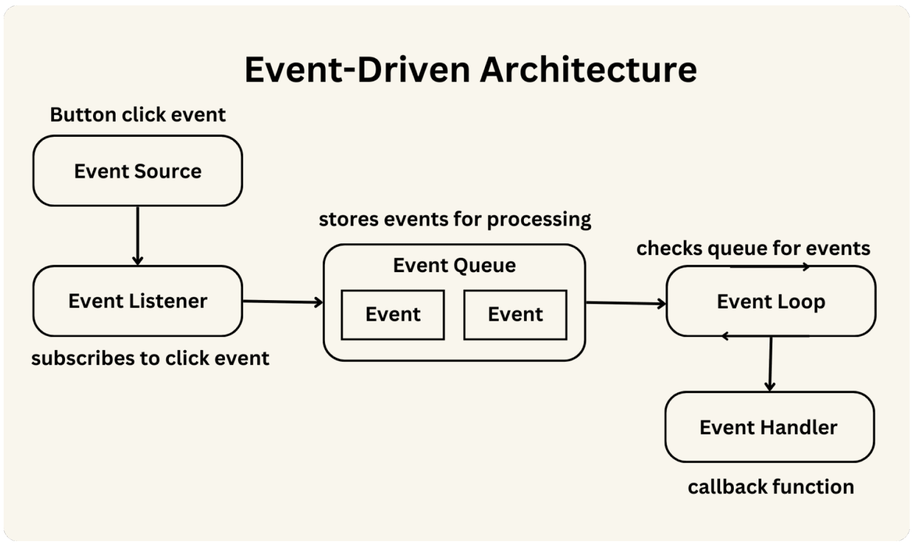
Use an Event-Driven Architecture (EDA) to ensure modularity, scalability, and responsiveness.

Build a frontend-only prototype in React + TypeScript with no backend dependency.

Design

Key Components:
- Event-Driven Architecture (EDA) enables decoupled communication via events.
- Built with React + TypeScript, no backend server (prototype phase).
- Events handled using mitt.js (lightweight event emitter).

Architecture Components:
- **Event Producers:** User actions (thread creation, replies, mentions).
- **Event Bus:** Central channel (eventBus.ts) using mitt.
- **Event Consumers:** UI components that respond to events (e.g., Notifications).

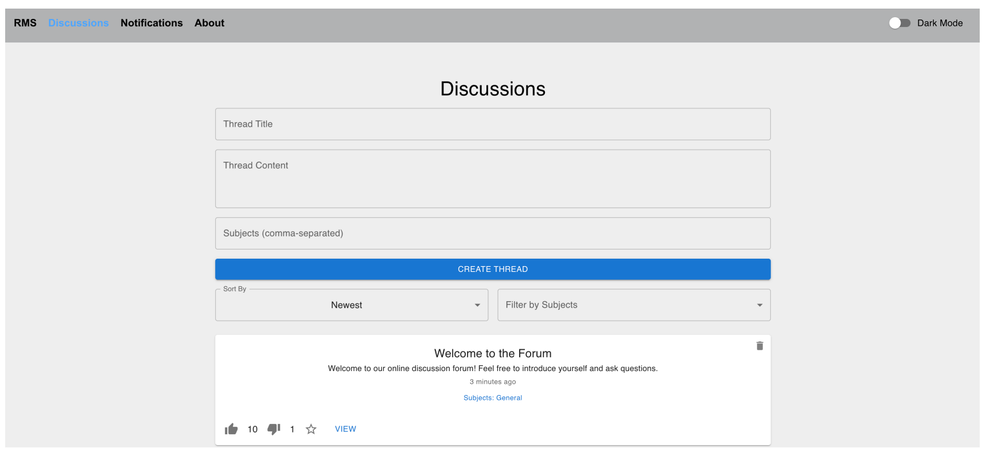


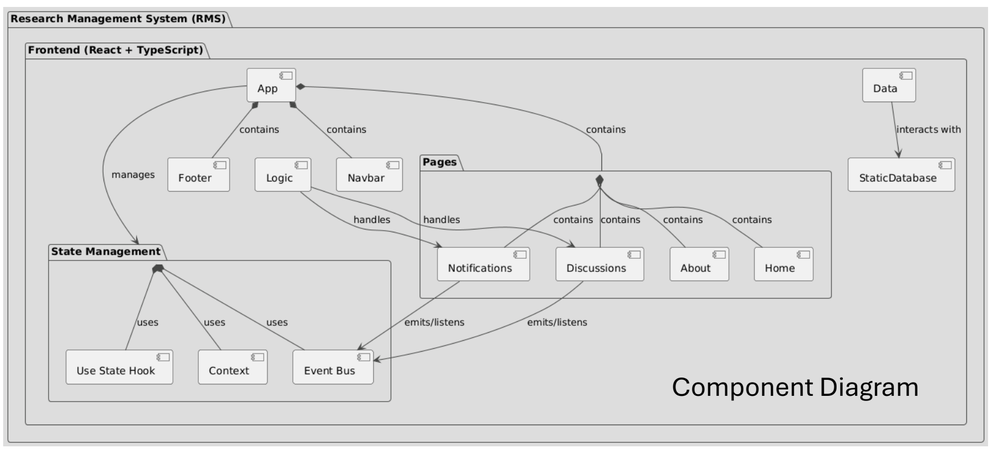
Prototype

The prototype of the RMS was built entirely using React with TypeScript. It follows an EDA that simulates real-time collaboration features without relying on a backend server.

Key functionalities implemented:
- Creating and replying to discussion threads
- @Mentions of users to simulate user engagement
- Subscribing to threads for notification updates
- Real-time updates handled via mitt.js (event emitter)

All events are routed through a centralized **eventBus.ts** file, which emits and listens for events such as newThread, mention and subscribedReply





Evaluation

Methods Used:
- **Lighthouse** – Evaluated performance, accessibility and UI responsiveness.
- **SonarQube** – Analysed code quality, maintainability and security risks.
- **ESLint** – Checked for React/TypeScript syntax errors and best practices.
- **ATAM** (Architecture Trade-off Analysis Method) – Assessed system architecture against key quality attributes.

Report

- **Performance:** 97% Lighthouse score. Responsive and optimized UI.
- **Usability:** Interface is intuitive with good accessibility structure.
- **Maintainability:** 28 minor issues found (SonarQube/ESLint), mainly related to code readability and best practices.
- **Security:** No backend—authentication and data security not implemented yet.

Challenges Faced:
- Difficulty simulating multi-user interaction without backend.
- Mock events created to demonstrate real-time event propagation.
- Managing asynchronous state across components using only frontend logic.
- Tight development timelines required fast, iterative prototyping.

GitHub Repository: https://github.com/rohansikder/RMS Live Demo (GitHub Pages): https://rohansikder.github.io/RMS
Contact: 24165816@studentmail.ul.ie